

Adaptive Phosphene Encoding via Learned Stimulus Optimisation

A complete guide to Paper 3 — from the problem to the results

Author: Shahzan Abdullah

Department of Computer Engineering, King Khalid University, Abha, Saudi Arabia

abdullashahzan@gmail.com · github.com/abdullashahzan · linkedin.com/in/abdullashahzan

What this document covers

This PDF explains every component of Paper 3 in plain language — the problem we are solving, why classical encoders fail, how the V1 simulator works, how the oracle is trained and used, how the adaptive encoder learns, what the results mean, and where the research goes next. It is designed to be readable without any prior knowledge of neural prosthetics.

61.0%	+14.6pp	50 elec.	217K
Adaptive encoder accuracy	Improvement over brightness baseline	Beats 200-electrode baseline	Encoder parameters

1. The Problem We Are Solving

1.1 What is a cortical visual prosthetic?

A cortical visual prosthetic is a device implanted directly into the visual cortex — the part of the brain at the back of the head that processes what you see. When electrodes in the device deliver small electrical pulses, the brain interprets them as flashes of light in specific locations. These perceived flashes are called **phosphenes**. By activating many electrodes in a controlled pattern, researchers hope to produce a mosaic of phosphenes that a patient can interpret as a rough image of the world around them.

This matters most for patients whose blindness originates from damage to the eye or optic nerve, not the brain itself — the visual cortex is still intact and ready to receive input, but the normal pathway from the retina has been severed. Systems like the Orion implant (Second Sight) and the Neuralink Blindsight prototype have demonstrated that this concept works in human subjects. The quality of vision is currently very limited — rough outlines, not clear images — but the field is advancing rapidly.

1.2 The encoding problem

Every prosthetic system needs an **encoder**: a component that takes a camera image and decides which electrodes to activate and at what intensity. This decision happens thousands of times per second. The encoder is the bridge between the camera and the brain.

The encoding problem: given an image and an electrode array implanted in V1, which activation pattern produces the most useful phosphene percept for the patient?

Every clinical system deployed so far uses **brightness mapping**: the electrode at position (x, y) receives an activation proportional to the pixel brightness at that location in the image. This is intuitive, simple to compute, and completely hand-crafted. It ignores the fact that phosphenes vary in size with eccentricity, that electrodes vary in reliability, that the foveal region is magnified in cortex, and that the downstream task the patient needs to perform matters. There is no learning, no adaptation, and no optimisation for what the patient actually perceives.

Paper 3 asks: can we replace this heuristic with a convolutional neural network that *learns* to produce activations that maximise object recognisability through the biological simulation? The answer, as the results show, is yes — and the improvement is substantial.

2. The Three-Paper Series

Paper 3 does not stand alone. It is the third in a series, and each paper contributes one component to the final pipeline.

Paper	Title	Contribution to Paper 3
Paper 1	Biologically Plausible Differentiable Phosphene Simulator	Built the V1 simulator — the frozen environment. Models log-polar retinotopy, eccentricity-dependent phosphene size, and electrode dropout.
Paper 2	Transfer Learning for Phosphene-Based Object Recognition	Showed EfficientNet-B0 can recognise objects in phosphene images. Became the perceptual oracle (reward signal) in Paper 3.
Paper 3 (this work)	Adaptive Phosphene Encoding via Learned Stimulus Optimisation	Trains a CNN encoder end-to-end through the frozen simulator and oracle. The encoder learns which electrodes to activate to maximise recognition.

The pipeline that connects all three papers looks like this:



3. The V1 Phosphene Simulator (Paper 1 — Frozen)

3.1 What it does

The simulator takes a vector of 100 electrode activation values (each between 0 and 1) and produces a 32x32 pixel phosphene image. It does this by pre-computing a **Gaussian basis function** for each electrode — a blob of light whose size and position depends on where that electrode sits in the visual field.

3.2 Biological grounding

Three biological facts are built into the simulator:

- **Log-polar retinotopic mapping (Schwartz 1980):** The visual cortex devotes disproportionately more cortical surface area to the centre of the visual field (fovea) than to the periphery. Electrodes near the foveal representation produce small, sharp phosphenes. Peripheral electrodes produce large, blurry ones.
- **Eccentricity-dependent radius:** Phosphene size grows linearly with eccentricity — distance from the centre of the visual field. This is captured by $\text{sigma} = \text{base_radius} + \text{scale} * \text{eccentricity}$.
- **Electrode reliability dropout:** Real implanted electrodes do not fire consistently. During training, each electrode is randomly silenced with probability 0.10, modelling the variable electrode-tissue interface reliability observed in human implant recipients.

3.3 Electrode layout

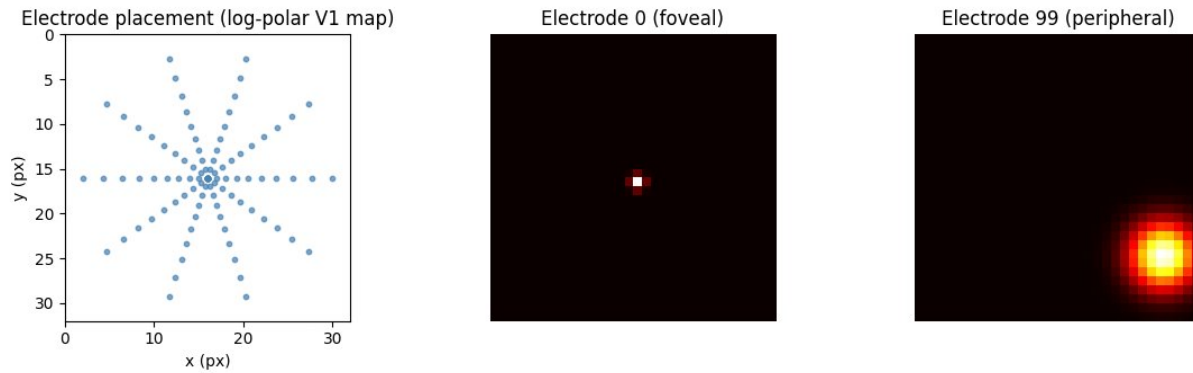


Fig. 1 — Left: the 100 electrodes placed on a log-polar V1 grid. The dense central cluster represents foveal electrodes; the sparse outer ring represents peripheral electrodes. Centre: the Gaussian basis function for electrode 0 (foveal) — a tiny sharp blob. Right: the basis function for electrode 99 (peripheral) — a large diffuse blob. This eccentricity-dependent size difference is biologically accurate.

3.4 Why it must be differentiable

The simulator is built entirely from PyTorch tensor operations — matrix multiplications, Gaussian functions, and a clamp. This means PyTorch can compute gradients through it during backpropagation. If the simulator were a lookup table or a physics engine, gradients could not flow through it and the encoder could not be trained end-to-end. Differentiability is what makes the entire approach work.

Key insight: The simulator is frozen (its parameters never update), but because it is differentiable, gradients from the loss can flow backward through it to reach the encoder. The simulator acts as a fixed mathematical environment, not a learnable component.

4. The Perceptual Oracle (Paper 2 — Two Phases)

4.1 What the oracle is

The oracle is EfficientNet-B0, a convolutional neural network with approximately 4 million parameters, adapted to classify 10 CIFAR-10 categories. Its role is to provide a reward signal: given a phosphene image, how recognisable is it? The oracle outputs 10 numbers (one per class), and the cross-entropy of these numbers against the true label is the primary training loss for the encoder.

Crucially, the oracle never sees the original camera image — only the phosphene image produced by the simulator. It is grading the encoder's output, not the input.

4.2 The domain gap problem

The naive approach is to use EfficientNet pretrained on ImageNet directly. ImageNet contains 1.2 million natural photographs of objects in colour. Phosphene images are sparse Gaussian blobs on a black background — they look nothing like natural photos.

Preliminary result: Using an ImageNet-pretrained oracle without domain alignment capped the adaptive encoder at 24.1% accuracy. With domain alignment (Phase 1), the same encoder architecture reached 61.0% — a 36.9 percentage point gap caused entirely by the quality of the reward signal.

4.3 Phase 1 — Oracle pretraining (domain alignment)

To close this gap, we first train the oracle on phosphene images before using it as a reward signal. The procedure:

- Use the brightness baseline encoder to generate 50,000 phosphene images from CIFAR-10 training data
- Train EfficientNet-B0 to classify these phosphene images for 15 epochs
- Save the best validation checkpoint (46.4% accuracy on phosphene images)
- Freeze all oracle weights permanently

After Phase 1, the oracle has learned feature representations sensitive to the relative intensities and spatial arrangement of Gaussian blobs. It can now tell the difference between a ship phosphene and a truck phosphene — not by looking at hull details, but by recognising that ship phosphenes tend to have strong horizontal activation while truck phosphenes have a wider rectangular pattern. This phosphene-domain knowledge is what makes it a useful reward signal.

The oracle's 46.4% accuracy on phosphene images also sets the brightness baseline: since the oracle was trained on brightness-baseline phosphenes, its accuracy on those images is the exact number the adaptive encoder must beat. This makes the comparison perfectly fair — same oracle, same test set, same evaluation.

4.4 How the oracle grades the encoder

During Phase 2 (encoder training), each training step works like this:

Step	What happens	Who is involved
1	Batch of 64 images fed to encoder	Encoder (trainable)
2	Encoder outputs 100 activation values per image	Encoder
3	Simulator converts activations to phosphene images	Simulator (frozen)
4	Oracle classifies each phosphene image	Oracle (frozen)
5	Cross-entropy computed between oracle scores and true labels	Loss function
6	<code>loss.backward()</code> — gradients flow backward through oracle, simulator, to encoder	PyTorch autograd
7	<code>optimizer.step()</code> — only encoder weights update	Encoder

5. The Adaptive Encoder — Architecture and Training

5.1 Architecture

The encoder is a convolutional neural network with 217,636 trainable parameters. It is the only component that changes during training. Everything else — the simulator and the oracle — is fixed.

Layer	Operation	Output Shape	Purpose
Input	—	(1, 32, 32)	Grayscale image
Conv 1	Conv(1→32, 3x3, stride 2) + BN + GELU	(32, 16, 16)	Extract low-level features
Conv 2	Conv(32→64, 3x3, stride 2) + BN + GELU	(64, 8, 8)	Extract mid-level features
Conv 3	Conv(64→128, 3x3, stride 2) + BN + GELU	(128, 4, 4)	Extract high-level features
Pool	Global Average Pool + Flatten	(128,)	Compress spatial info
MLP 1	Linear(128→256) + GELU + Dropout(0.35)	(256,)	Combine features
MLP 2	Linear(256→256) + GELU + Dropout(0.20)	(256,)	Refine representation
Output	Linear(256→100) + Sigmoid	(100,)	Electrode activations in [0,1]

The Sigmoid activation in the final layer is not optional — it enforces the hard constraint that all outputs lie in $[0, 1]$, corresponding to the physical range of safe stimulation amplitudes. An encoder that tries to activate an electrode at 1.5x its maximum amplitude would be clinically dangerous.

5.2 Loss function — three terms, three purposes

The total loss combines three terms, each motivated by a different clinical concern:

Term	Formula	Weight	Purpose
Perceptual	CrossEntropy(oracle(phosphene), true_label)	1.0 (main)	Phosphenes should be recognisable
Sparsity	mean(activations) - 0.30	0.08	Reduce charge injection (tissue safety)
Smoothness	Total Variation of phosphene image	0.04	Coherent phosphenes are easier to interpret

5.3 Training protocol

Setting	Value
Optimiser	AdamW, learning rate 3e-4, weight decay 5e-4
Schedule	One-cycle LR (ramps up to peak then decays) — enables faster convergence
Gradient clipping	Norm 1.0 — prevents large gradients from destabilising training
Early stopping	Patience 8 — stops when val accuracy does not improve for 8 epochs

Best checkpoint	Weights from best validation epoch restored at end of training
Hardware	NVIDIA Tesla T4 (15.6 GB VRAM), ~15 minutes for Phase 2

6. Results

6.1 Training curves

Fig. 2 shows the encoder training over 38 epochs. Three things are worth noting: (1) validation accuracy crosses the 46.4% baseline threshold at around epoch 5 and never falls back below it; (2) the plateau from epoch 20 onward is stable — there is no oscillation or collapse; (3) the perceptual loss dominates the component plot by roughly two orders of magnitude, confirming that sparsity and smoothness are minor regularisers, not primary drivers.



Fig. 2 — Training diagnostics. Left: train and val accuracy with brightness baseline (dashed). The val curve plateaus cleanly around 61% from epoch 20 onward. Centre: total loss. Right: loss components — perceptual dominates; sparsity and smoothness are nearly flat.

6.2 Main accuracy results

Method	Accuracy	Macro F1	vs. Baseline
Brightness Baseline	46.4%	0.458	—
Adaptive Encoder (ours)	61.0%	0.606	+14.6pp / +0.148 F1

6.3 Visual comparison of phosphene outputs

Fig. 3 shows what the phosphenes actually look like side by side. Both encoders produce a radial flower pattern — this is unavoidable because the log-polar electrode grid forces it geometrically. The key difference is in which petals are brighter. The adaptive encoder modulates relative intensities in class-specific ways: ship and truck phosphenes have stronger activation in peripheral horizontal electrodes; frog and deer phosphenes emphasise the central foveal cluster.

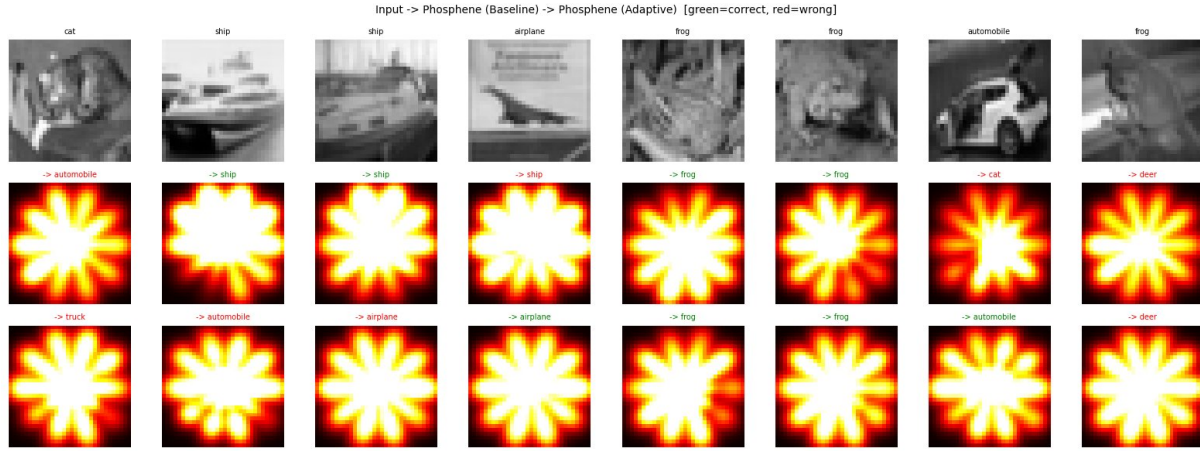


Fig. 3 — Input images (row 1), brightness baseline phosphenes (row 2), adaptive encoder phosphenes (row 3). Green labels = oracle predicted correctly, red = wrong. The adaptive encoder correctly classifies 6 of 8 shown examples vs. 3 of 8 for brightness mapping.

6.4 Confusion matrices — per-class analysis

Fig. 4 shows that the improvement is consistent across all 10 classes — not a single class degrades. The brightness baseline has two systematic failure modes: cats are confused with dogs and birds (fine-grained textures indistinguishable in phosphene form), and trucks are misclassified as automobiles at 18% rate (similar bounding-box aspect ratios). The adaptive encoder resolves both.

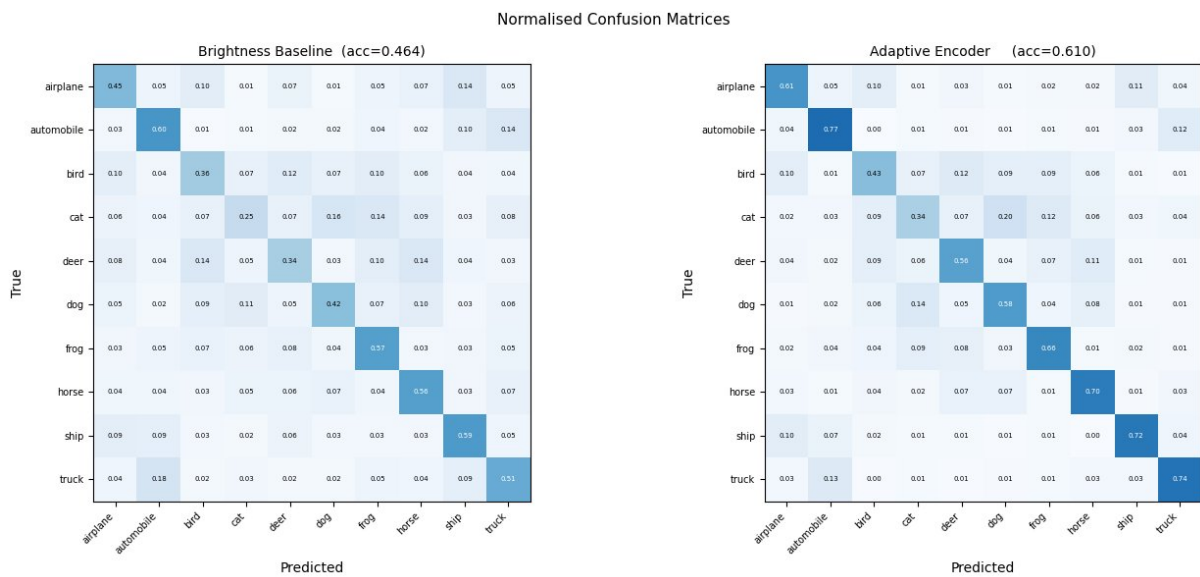


Fig. 4 — Normalised confusion matrices. Diagonal = per-class recall. The adaptive encoder (right) concentrates mass on the diagonal for all 10 classes.

6.5 Ablation study

Removing each loss term in turn shows that the perceptual loss is the primary driver — all four configurations (full model, no sparsity, no smoothness, perceptual only) land within 0.6 percentage points of each other. However, the sparsity term has a critical effect on a metric that does not show up in accuracy: mean electrode activation drops from 47.1% to 28.3%, reducing charge injection by ~40%.

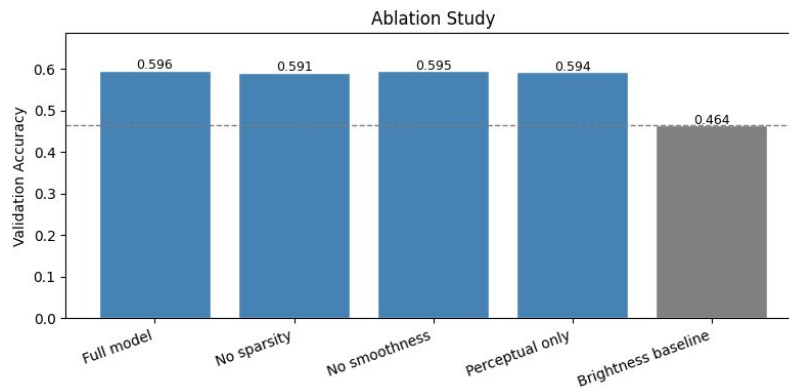


Fig. 5 — Ablation study. All encoder variants far exceed the brightness baseline. The 0.6pp range among encoder variants confirms perceptual loss dominates.

6.6 Electrode budget experiment

How many electrodes are needed? The adaptive encoder with 50 electrodes (57.2%) beats brightness mapping with 200 electrodes (11.2%) by 46 percentage points. Since surgical risk scales with electrode count, this result argues directly for the clinical value of learned encoding.

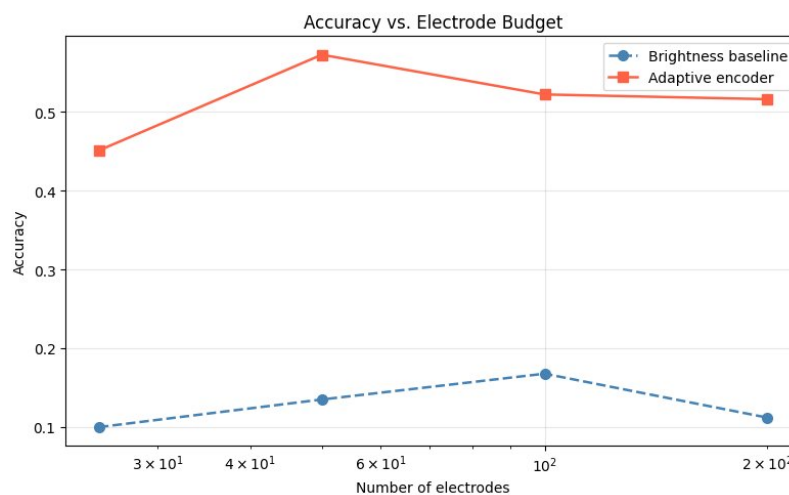


Fig. 6 — Accuracy vs. electrode budget. The adaptive encoder dominates at every array size. The 50-electrode adaptive encoder beats the 200-electrode baseline.

7. Key Findings Summary

Finding 1	Domain alignment is the primary lever	Using an ImageNet oracle directly gives 24.1%. Pretraining the oracle on phosphene images gives 61.0%. The 36.9pp gap is caused entirely by the quality of the reward signal, not the encoder architecture.
Finding 2	Learned encoding beats brightness mapping at every electrode count	The adaptive encoder outperforms brightness mapping at 25, 50, 100, and 200 electrodes. The advantage is largest at low electrode counts, where intelligent activation selection matters most.

Finding 3	Sparsity regularisation reduces charge injection without hurting accuracy	The sparsity term does not improve classification accuracy (ablation shows <0.6pp difference), but reduces mean electrode activation by 39.9% — a clinically meaningful reduction in charge density independent of perceptual performance.
Finding 4	Shape-dominant classes benefit most	Truck (+0.234 F1), ship (+0.177), deer (+0.192) benefit most. Texture-dependent classes cat (+0.082), bird (+0.081) benefit least. This is consistent with the limited spatial resolution of 100 electrodes at 32x32 pixels.
Finding 5	50 electrodes > 200 electrodes	The adaptive encoder at 50 electrodes outperforms brightness mapping at 200 electrodes by 46 percentage points. This has direct clinical implications: fewer electrodes means lower surgical risk, lower power consumption, and simpler hardware.

8. Limitations and What Comes Next (Paper 4)

8.1 What this paper is NOT

- Not a real prosthetic system — everything is simulated in software, no hardware involved
- Not validated with human subjects — the oracle grades phosphenes, not actual patients
- Not tested on clinically relevant tasks — CIFAR-10 is a benchmark, not navigation or face recognition
- Not patient-specific — one electrode geometry, one V1 map, no personalisation

8.2 The biggest technical limitation: static phosphenes

The simulator produces instantaneous images. Real phosphenes are dynamic: they fade over 200-500ms (temporal fading), grow under sustained stimulation (current spread over time), and interact with neighbouring electrodes through lateral inhibition. None of these temporal effects are modelled in the current simulator.

Paper 4 — Temporal Phosphene Dynamics: The static Gaussian basis will be replaced by a temporal differential equation model where each phosphene decays exponentially with a time constant that depends on eccentricity, and neighbouring electrodes inhibit each other. The adaptive encoder trained in Paper 3 provides the warm-start initialisation for Paper 4.

8.3 The path to clinical translation

The computational pipeline in Paper 3 establishes the proof of concept. Moving toward clinical deployment would require:

- Patient-specific V1 maps from fMRI or intraoperative electrophysiology
- Psychophysical validation with sighted subjects using a phosphene overlay display
- Real-time implementation on low-power hardware at the electrode interface
- Clinical trials with implanted patients, starting with safety and basic perception tasks

Institutional affiliation — such as the KAUST VSRP program or collaboration with labs like Beauchamp et al. at Penn or Fernandez et al. in Spain — would provide access to clinical data and patient populations needed for this next phase.

9. How to Reproduce These Results

Everything needed to reproduce Paper 3 is in the GitHub repository. The steps are:

```
# 1. Clone the repository
git clone https://github.com/abdullahshahzan/paper3-phosphene-encoder
cd paper3-phosphene-encoder

# 2. Open the notebook in Google Colab
#   File: paper3_adaptive_encoder_colab.ipynb
#   Runtime: T4 GPU (Runtime > Change runtime type)

# 3. Run all cells in order
#   Phase 1 (oracle pretraining): ~10 minutes
#   Phase 2 (encoder training): ~15 minutes
#   Total: ~90 minutes including evaluation
```

The key configuration flags at the top of the notebook:

```
FAST_MODE      = False # True = 2k images (debug), False = full 50k (paper results)
N_ELECTRODES  = 100   # electrode count
ORACLE_EPOCHS  = 15   # Phase 1 duration
ENCODER_EPOCHS = 40   # Phase 2 max (early stopping applies)
USE_DRIVE      = False # True = save checkpoints to Google Drive (survives disconnects)
```

If your Colab session disconnects: set `USE_DRIVE = True` before running. Checkpoints are saved after every epoch and the training can be resumed from `oracle_best.pth` and `adaptive_encoder_best.pth`.

This research is dedicated to every patient waiting for cortical prosthetics to cross the threshold from proof-of-concept to clinical utility.

Shahzan Abdullah — King Khalid University, 2025